

II-Les opérateurs

Le langage C est connu pour la richesse de ses opérateurs. Il apporte aussi quelques notions innovantes en matière d'opérateurs. Les opérateurs sont les éléments du langage qui permettent de faire du calcul ou de définir des relations.

Ils servent à combiner des variables et des constantes pour réaliser des expressions et d'agir sur le contenu de variables. Ils effectuent un calcul numérique (opérateurs mathématiques), fournissent une condition logique pour un test (opérateurs logiques et relationnels), agissent sur certains bits d'une variable (opérateurs de manipulation de bits) ou réalisent des opérations spécifiques (affectation, conversions, obtention d'une adresse, etc).

II-1 Les opérateurs arithmétiques :

Ils s'appliquent à tous les types numériques (int, *double*...), à l'exception de l'opérateur **modulo** qui ne concerne que les entiers.

Opérateur	Rôle	Exemples
+	addition	2+3 vaut 5
-	soustraction	2-3 vaut -1
*	produit	2*3 vaut 6
/	quotient (entier ou réel !)	11./2. vaut 5.5 (division réelle) mais 7/3 vaut 2 (division entière) !
%	modulo (reste de la division entière)	11%3 vaut 2 24%8 vaut 0 <i>Très utile en informatique</i>

Les opérateurs mathématiques

REMARQUE

- 1- les opérateurs binaires, c'est-à-dire agissant sur deux opérandes, ne sont a priori définis que pour des opérandes de même type et ils fournissent un résultat de ce type. Par exemple, 5./2. est le quotient de deux valeurs de type *double* et l'opérateur quotient / fournit le résultat 2.5 de type *double*. Par contre, 5/2 est le quotient de deux entiers et le résultat est l'entier 2 !
- 2- Quand les deux opérandes ne sont pas du même type, une opération de conversion implicite est mise en œuvre par le compilateur afin que le calcul soit fait dans le **type dominant**. La hiérarchie des types est :
char < short int < long int < float < double
- 3- Quand plusieurs opérateurs apparaissent dans une même expression, les règles traditionnelles de **priorité** de l'algèbre s'appliquent (voir tableau des priorités) : d'abord les opérateurs unaires + et -, puis les opérateurs *, /, et %, puis enfin opérateurs binaires + et -. Des parenthèses permettent de s'affranchir des priorités.

Exemple01 : Opérateurs mathématiques

$res = 5 + 9 / 4$	$9 / 4$ qui vaut 2 est ajouté à 5 → res vaut 7 au final.
$res = (5 + 9) / 4$	$5 + 9$ qui vaut 14 est divisé par 4 (division entière) → res vaut 3 au final.
$res = (5 + 9.) / 4$	$5 + 9.$ qui vaut 14. (conversion implicite en <i>double</i> de 5 et résultat <i>double</i>) est divisé par 4 (division réelle) → res vaut 3.5 au final.
$res = 4 * 2 + 9 \% 4$	$4 * 2$ qui vaut 8 est ajouté à $9 \% 4$ qui vaut 1 → res vaut 9 au final.
$i = (i + 1) \% 10$	permet d'incrémenter i « modulo 10 » : i prend successivement les valeurs 0, 1, 2, ..., 8, 9, 0, 1...

C propose également des opérateurs spéciaux d'incrément ou décrémentation

++	Incrément
--	décrément

A la différence des autres, ces opérateurs ne sont applicables qu'à des variables et non à des expressions. Ecrire : **n++** est licite et incrémente la valeur de la variable **n** de 1, alors qu'écrire **5++** n'a aucun sens.

Lorsqu'une variable fait l'objet d'une incrément ou décrémentation et, en même temps, apparaît dans une expression, ces opérateurs s'utilisent en **préfixe** (**++n**) ou **postfixe** (**n++**).

incrémenter/décrémenter une variable et en même temps affecter sa valeur à une autre variable. Dans ce cas, nous devons choisir entre la notation préfixe et postfixe :

X = n++	passer d'abord la valeur de I à X et <i>incrémenter après</i>
X = n--	passer d'abord la valeur de I à X et <i>décrémenter après</i>
X = ++n	<i>incrémenter d'abord</i> et passer la valeur incrémentée à X
X = --n	<i>décrémenter d'abord</i> et passer la valeur décrémentée à X

II-1-1 L'opérateur d'affectation = (« reçoit »)

L'affectation permet de « ranger » une valeur dans un emplacement mémoire (en général une variable) Mais l'affectation n'est pas seulement un traitement à effectuer : c'est aussi une **expression** qui prend comme valeur la valeur affectée. Cela permet d'utiliser l'affectation comme opérande d'une autre expression. En voici deux **Exemples02** :

min = max = 0 ;

if ((val=getchar())=='q') ... /* ici les parenthèses sont indispensables */

Pour comprendre ce qui se passe, commençons par deux cas simples.

L'instruction **nombre=4** est une expression qui :

- place la valeur 4 dans la variable *nombre* : c'est l'affectation de *nombre* ;
- prend elle-même la valeur 4.

De même, l'écriture **max=5*2+3** est une expression qui :

- évalue l'expression à droite (13) et donne à la variable *max* la valeur 13 ;
- prend elle-même la valeur 13 ;

- L'expression **max=5*2+3** peut alors se placer à droite d'une autre affectation. Par exemple, on peut écrire : **min=max=5*2+3** → les variables **min** et **max** auront toutes deux la valeur 13.

Remarquons que pour une fois, l'associativité se fait **de la droite vers la gauche**. Ainsi lorsqu'on écrit **a=b=c=0**, l'expression la plus à droite est évaluée (elle vaut 0), puis sa valeur est affectée à la variable **c** ; en même temps, l'expression **(c=0)** prend la valeur 0, qui est ensuite affectée à la variable **b** ; l'expression **(b=c=0)** prend alors la valeur 0, etc ...

L'association d'un opérateur arithmétique avec l'opérateur d'affectation donne les opérateurs décrits dans le tableau suivant qui utilise chacun des opérations arithmétiques et les façons de lire les différentes expressions, en supposant que le type entier est représenté sur 32 bits, et que les deux variables **i** et **j** sont définies de la manière suivante : **int i = 100, j = 5;**

expression	résultat	équivalence lecture
opérateurs arithmétiques		
i += 10	110	i = i + 10 ajoute 10 à i
i += j	115	i = i + j ajoute j à i
i -= 5	110	i = i - 5 retranche 5 à i
i -= j	105	i = i - j retranche j à i
i *= 10	1050	i = i * 10 multiplie i par 10
i *= j	5250	i = i * j multiplie i par j
i /= 10	525	i = i / 10 divise i par 10
i /= j	105	i = i / j divise i par j
i %= 10	5	i = i % 10 i reçoit le reste de la division entière de i par 10

II-2 Opérateurs logiques : ET OU NON

Ces opérateurs logiques permettent de relier entre elles plusieurs conditions pour réaliser une condition **multiple**. Ils sont souvent utilisés en association avec les opérateurs relationnels. Opérateurs logiques et relationnels ont en commun de fournir un résultat sous la forme d'une **valeur entière (0 si faux, 1 si vrai)**.

Le Langage C possède trois opérateurs logiques :

Opérateur	Signification
&&	ET logique (résultat 0 ou 1)
 	OU logique (résultat 0 ou 1)
!	NON logique (résultat 0 ou 1)

Les opérateurs logiques acceptent comme opérandes toutes les valeurs numériques (réels compris !). Ils considèrent que :

- 0 correspond à *faux* ;
- toute valeur **non nulle** (et pas seulement 1) correspond à *vrai*. -4.76 est considéré comme *vrai*...

Exemple03 : Expressions avec opérateurs logiques

a<b && c<d	prend la valeur 1 (vrai) si les deux conditions a<b et c<d sont toutes les deux vraies (de valeur non nulle), la valeur 0 (faux) dans le cas contraire.
a<b c==0	prend la valeur 1 (vrai) si l'une au moins des deux expressions a<b et c==0 est vraie (de valeur non nulle), la valeur 0 (faux) dans le cas contraire.
!(a<b)	prend la valeur 1 (vrai) si la condition a<b est fausse (de valeur 0), la valeur 0 (faux) dans le cas contraire. Cette expression est équivalente à a>=b.

II-3 Les opérateurs relationnels : inférieur, *supérieur*, *égal*, *différent*...

Souvent associés aux opérateurs logiques, ils permettent de **comparer** des expressions pour effectuer des **tests**, fréquents en informatique.

Exemples04 : a < 100 a == b a >= b a != b

Contrairement aux autres langages, le résultat en Langage C d'un opérateur relationnel ou logique n'est pas une valeur booléenne (vrai ou faux), mais un entier qui vaut :
• 0 si le résultat de la comparaison est faux ;
• 1 si le résultat de la comparaison est vrai .

Opérateur	Signification
<	inférieur à
<=	inférieur ou égal à
>	supérieur à
>=	supérieur ou égal à
==	égal à ATTENTION à ne pas confondre avec = qui est l'affectation !
!=	différent de

Exercice 1 :

Evaluer les expressions suivantes en supposant

a=20 b=5 c=-10 d=2 x=12 y=15

Notez chaque fois la valeur rendue comme résultat de l'expression et les valeurs des variables dont le contenu a changé.

- (1) (5*X)+2*((3*B)+4)
- (2) (5*(X+2)*3)*(B+4)
- (3) A == (B=5)
- (4) A += (X+5)
- (5) A != (C * = (-D))
- (6) A *= C+(X-D)
- (7) A %= D++
- (8) A %= ++D

- (9) $(X++)*(A+C)$
 (10) $A = X*(B<C)+Y*!(B<C)$
 (11) $!(X-D+C)||D$
 (12) $A\&\&B||!0\&\&C\&\&!D$
 (13) $((A\&\&B)||(!0\&\&C))\&\&!D$
 (14) $((A\&\&B)||!0)\&\&(C\&\&(!D))$

II-4 Opérateurs spéciaux

C dispose de quelques opérateurs un peu exotiques qui n'ont rien de fondamental mais peuvent rendre service dans un certain nombre de cas.

II-4-1 L'opérateur *sizeof* (taille en octets)

Pour rendre un programme portable et pour s'éviter des calculs fastidieux, il ne faut jamais écrire "en dur" le nombre d'octets occupés par une variable : l'opérateur *sizeof* est là pour ça. *sizeof* fournit la taille (en octets) du type ou de la variable qui suit. Sa syntaxe est :

sizeof (type ou nom de variable)

Exemple06 : Opérateur *sizeof*

int n ;	sizeof(n) peut valoir 2 ou 4 selon l'ordinateur et le logiciel.
double x ;	sizeof(x) vaut 8
	sizeof(float) vaut 4
	sizeof(char) vaut 1

II-4-2 L'opérateur d'adresse &

Le seul moyen de connaître l'adresse mémoire d'une variable est d'utiliser l'opérateur de calcul d'adresse **&** :

&var

représente l'adresse de la variable *var* (l'adresse du premier octet de la zone mémoire qu'elle occupe). L'opérateur d'adresse est très utile comme argument d'appel pour les fonctions qui utilisent un passage en paramètre par adresse.

Exemple07 :

scanf("%lf", &toto) ;

II-5 Priorité des opérateurs dans l'ordre décroissant

Voici la priorité des opérateurs en commençant par les plus prioritaire

Opérateurs	Rôle	Exemples
appel de fonction accès aux éléments d'un tableau ou d'une structure		fct() tab[i] toto.champ

! ~ ++ -- * &(adr.) (cast) sizeof -(unaire)	divers op. unaires	&toto i++ -a (double) *ptr
* / % + -	op. arithmétiques (binaires)	a%b a+b
<< >>	op. bit de décalage	1<<5
< <= > >= == !=	op. relationnels	a<b a != b
& ^ 	op. bit (binaires)	a & 0xF0 a ^ 0xF0 a 0xF0
&& 	op. logiques (binaires)	cond1 && cond2 cond1 cond2
? :	affectation condit.	res= a>0 ? 1 : 0
= += -= *= /= %= &= = <<= >>=	affectation	res = 67

III Les fonctions arithmétiques standard :

Les fonctions suivantes sont prédéfinies dans la bibliothèque standard `<math>`. Pour pouvoir les utiliser, le programme doit contenir la ligne :

#include <math.h>

La liste de fonctions suivantes ne cite que les fonctions les plus courantes.

III-1 Fonctions arithmétiques :

<i>COMMANDE C</i>	<i>EXPLICATION</i>	<i>LANG. ALGORITHMIQUE</i>
exp(X)	Fonction exponentielle	e^X
log(X)	Logarithme naturel	ln(X) , X>0
log10(X)	Logarithme à base 10	log₁₀(X) , X>0
pow(X,Y)	X exposant Y	X^Y
sqrt(X)	Racine carrée de X	pour X>0
fabs(X)	Valeur absolue de X	 X
floor(X)	Arrondir en moins	int(X)
ceil(X)	Arrondir en plus	
fmod(X,Y)	Reste rationnel de X/Y (même signe que X)	pour X différent de 0

sin(X), cos(X), tan(X)	sinus, cosinus, tangente de X
asin(X), acos(X), atan(X)	arcsin(X), arccos(X), arctan(X)
sinh(X), cosh(X), tanh(X)	sinus, cosinus, tangente hyperboliques de X

